

C++ Object Oriented Programming

Assignment

Instructor: Hafiz Muhammad Usman

Due Date: 20-05-2026

Submission Format: Printed (hard copy) only. Both theory answers and practical work (source code + output) must be printed and submitted.

Instructions

- Write your **Name**, **Roll Number**, and **Section** clearly on the top of the first page.
 - Answer all theory questions in your own words.
 - For practical tasks, provide the complete C++ code and a printed screenshot of the program output.
 - Plagiarism will result in deduction of marks.
-

A. THEORY SECTION (30 Marks)

1. Constructors and Overloaded Constructors (5 marks)

Define a constructor in C++. Explain the difference between a **default constructor** and a **parameterized constructor**. Why might a class need more than one constructor? Give a small code snippet to illustrate overloaded constructors.

2. Default Copy Constructor (5 marks)

What is a copy constructor? Under what circumstances is the compiler-generated default copy constructor invoked? Explain the concept of **shallow copy** and discuss the major drawback of the default copy constructor when a class contains pointer members.

3. Destructors (5 marks)

What is a destructor? Describe its syntax and explain when and why a destructor becomes essential. Provide a realistic scenario (with a brief code outline) where a user-defined destructor prevents a resource leak.

4. Static Class Data (5 marks)

What is a static data member in a C++ class? How is it different from a non-static data member? Write a simple example showing the declaration, definition, and usage of a static member. Why would you use a static member instead of a global variable?

5. Inheritance – Basics and Access Specifiers (5 marks)

Define inheritance. Differentiate between a base class and a derived class. How does the `protected` access specifier influence member accessibility in a derived class? Give an example showing which base class members can be accessed directly in a publicly derived class.

6. Types of Inheritance (public, protected, private) (5 marks)

Summarize the effect of **public**, **protected**, and **private** inheritance on the accessibility of base class members in the derived class and in further derived classes. Present your answer in the form of a small table or clearly structured sentences.

B. PRACTICAL SECTION (70 Marks)

Practical 1 – Constructors, Copy Constructor, and Destructor (25 marks)

Write a C++ program that defines a class `Book` with the following attributes:

- `title` (string)
- `author` (string)
- `price` (double)

Implement the following member functions:

1. **Default constructor** – initialises `title` and `author` to "Unknown" and `price` to 0.0. Display a message "Default constructor called".
2. **Parameterised constructor** – takes all three values as arguments and initialises the attributes. Display "Parameterised constructor called".
3. **Copy constructor** – performs a member-wise copy (deep copy, no pointers here) and displays "Copy constructor called".
4. **Destructor** – displays "Book object destroyed: <title>" (use the actual title).
5. **Display function** – prints the book details in a neat format.

In `main()`:

- Create an object using the default constructor.
- Create another object using the parameterised constructor with values of your choice.
- Create a third object by copying the parameterised object using the copy constructor.
- Call the `display()` function on all three objects.
- Observe and note the **order of constructor and destructor calls** (add output messages accordingly).

Submission: Printed source code and output.

Practical 2 – Static Data Member (20 marks)

Implement a class `Student` with:

- Private data members: `name` (string) and `rollNo` (int).
- A **static data member** `count` that tracks the total number of `Student` objects currently in existence.
- Increment `count` in **every constructor** and decrement it in the **destructor**.
- A **static member function** `getCount()` that returns the value of `count`.

In `main()`:

- Display the initial value of `getCount()`.
- Create three `Student` objects. Display the count again.
- Inside a `{}` block, create two more `Student` objects and display the count inside the block.
- After the block ends, display the count one more time to show that the destroyed objects were decremented.

Explain in comments why the count can be accessed without any instance and why it retains its value across all objects.

Submission: Printed source code and output.

Practical 3 – Inheritance: public, protected, and private (25 marks)

Design a class hierarchy that demonstrates **all three types of inheritance** along with base/derived constructors and member access rules.

Classes required:

1. `Vehicle` (base class)
 - `protected` data member: `maxSpeed` (int)
 - Public constructor that initialises `maxSpeed`
 - Public member function `showSpeed()` that prints the max speed.
2. `Car` – publicly inherited from `Vehicle`
 - Additional private member: `brand` (string)
 - Constructor that takes `speed` and `brand`, passes `speed` to `Vehicle` constructor
 - Public function `display()` that calls `showSpeed()` and prints `brand`.
3. `SportsCar` – **protected** inheritance from `Car`
 - Additional private member: `turbo` (bool)
 - Constructor that initialises all data (call base class constructor)
 - Public function `showSportsSpecs()` that internally calls `showSpeed()` and prints turbo status.
 - *Note:* In `SportsCar`, `showSpeed()` and `Car`'s public members have become **protected**, so they are accessible inside `SportsCar` but **not** from `main()`.
4. `LuxurySportsCar` – publicly inherited from `SportsCar`
 - Additional private member: `interior` (string)
 - Constructor that initialises all data
 - Public function `showLuxuryDetails()` that calls `showSpeed()` (still accessible as `protected`) and prints `interior`.
 - *Observe:* `showSpeed()` remains accessible in further derived classes but still not outside the hierarchy.
5. `Truck` – **private** inheritance from `Vehicle`
 - Additional private member: `loadCapacity` (double)
 - Constructor that initialises `maxSpeed` (via `Vehicle`) and `loadCapacity`
 - Public function `truckInfo()` that calls `showSpeed()` (allowed inside the class) and prints capacity.
 - *Note:* After private inheritance, all inherited members become **private** in `Truck`, so any further derived class from `Truck` **cannot** access `Vehicle` members.

6. **(Optional)** HeavyTruck – publicly inherited from Truck

- Try to call showSpeed() inside its member function and **comment the line** explaining why it causes a compilation error.

In main():

- Create a Car object and call display() – works.
- Create a SportsCar object and call showSportsSpecs() – works.
- Try to call showSpeed() directly on a SportsCar object – put the line in comment and explain why it fails (protected inheritance).
- Create a LuxurySportsCar object and call its function.
- Create a Truck object, call truckInfo().
- Try calling showSpeed() on a Truck object directly – comment and explain (private inheritance).
- Show the compilation error for HeavyTruck (comment it out after testing).

Important: Insert clear comments throughout the code explaining *why* certain members are accessible or inaccessible according to the inheritance type used. Mention the role of the protected access specifier.

Submission: Printed source code and output (compile and run the parts that compile; comment out lines that produce errors and add explanatory comments).

Marking Distribution

Section	Marks
Theory (6 questions)	30
Practical 1	25
Practical 2	20
Practical 3	25
Total	100

